664 HW2

Alex Caramanico

2025-03-12

Load Libraries

```
library(ggplot2)
library(tidyverse)
library(cowplot)
library(norm)
library(rstan)
library(bayesplot)
```

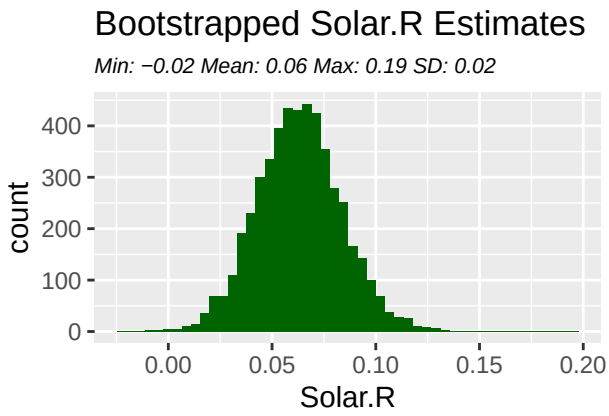
Lecture 4 Problems

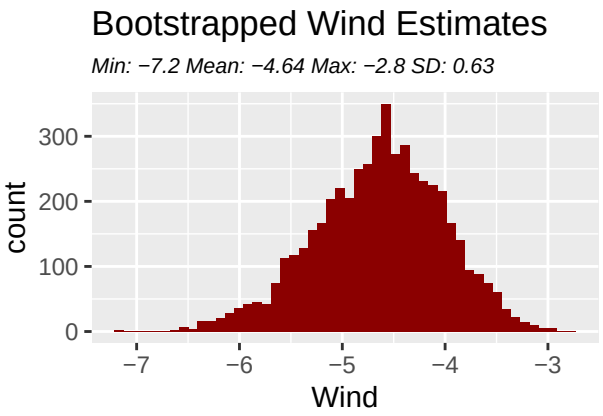
1.

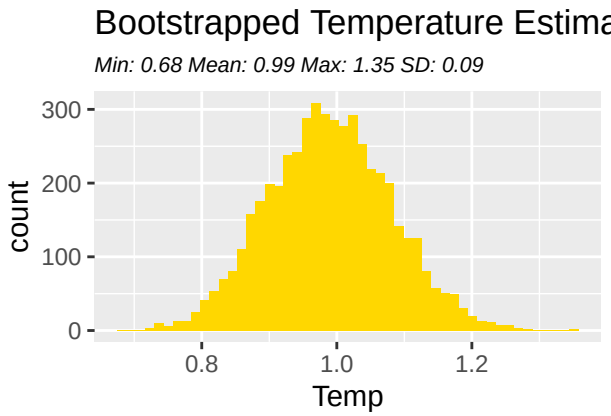
From your real dataset run a regression analysis using complete data (no missing values) using your preferred software; and then try to get the bootstrap estimates of the regression and compare with them (e.g., sample code can be found in our lecture).

```
data("airquality")
complete_data <- na.omit(airquality)

### Base Regression
base_model <- lm(Ozone~Solar.R + Wind + Temp-1,data=complete_data) %>% summary()

### Bootstrapping, Summary Statistics
# sample() rows from complete data, still want same n, repeat regression
boots <- 5000
parameter_estimates <- matrix(0,ncol=3,nrow=boots)
median_ozone <- rep(0,boots)
for (i in 1:boots){
  rows <- sample(1:nrow(complete_data),nrow(complete_data),replace = T)
  boot_data <- complete_data[rows,]
  model_fit <- lm(Ozone~Solar.R + Wind + Temp-1,data=boot_data)
  parameter_estimates[i,] <- model_fit$coefficients
  median_ozone[i] <- quantile(boot_data[,1],0.5)
}
colnames(parameter_estimates) <- c("Solar.R","Wind","Temp")
summary_stats <- rbind(apply(data.frame(parameter_estimates),2,summary) %>% round(2),
                       SD=apply(data.frame(parameter_estimates),2,sd) %>% round(2))
```

```r
### Plotting Results, Comparisons
solar.r <- ggplot(data=parameter_estimates) +
  geom_histogram(mapping=aes(x=Solar.R),bins=50,fill='darkgreen') +
  ggtitle("Bootstrapped Solar.R Estimates",
          paste("Min:",summary_stats[,1][1],
                "Mean:",summary_stats[,1][4],
                "Max:",summary_stats[,1][6],
                "SD:",summary_stats[,1][7],sep=" ")) +
  theme(plot.subtitle = element_text(size = 8, face = "italic"))

wind <- ggplot(data=parameter_estimates) +
  geom_histogram(mapping=aes(x=Wind),bins=50,fill='darkred') +
  ggtitle("Bootstrapped Wind Estimates",
          paste("Min:",summary_stats[,2][1],
                "Mean:",summary_stats[,2][4],
                "Max:",summary_stats[,2][6],
                "SD:",summary_stats[,2][7],sep=" ")) +
  theme(plot.subtitle = element_text(size = 8, face = "italic"))

temp <- ggplot(data=parameter_estimates) +
  geom_histogram(mapping=aes(x=Temp),bins=50,fill='gold') +
  ggtitle("Bootstrapped Temperature Estimates",
          paste("Min:",summary_stats[,3][1],
                "Mean:",summary_stats[,3][4],
                "Max:",summary_stats[,3][6],
                "SD:",summary_stats[,3][7],sep=" ")) +
  theme(plot.subtitle = element_text(size = 8, face = "italic"))

plot_grid(solar.r,wind,temp)
```





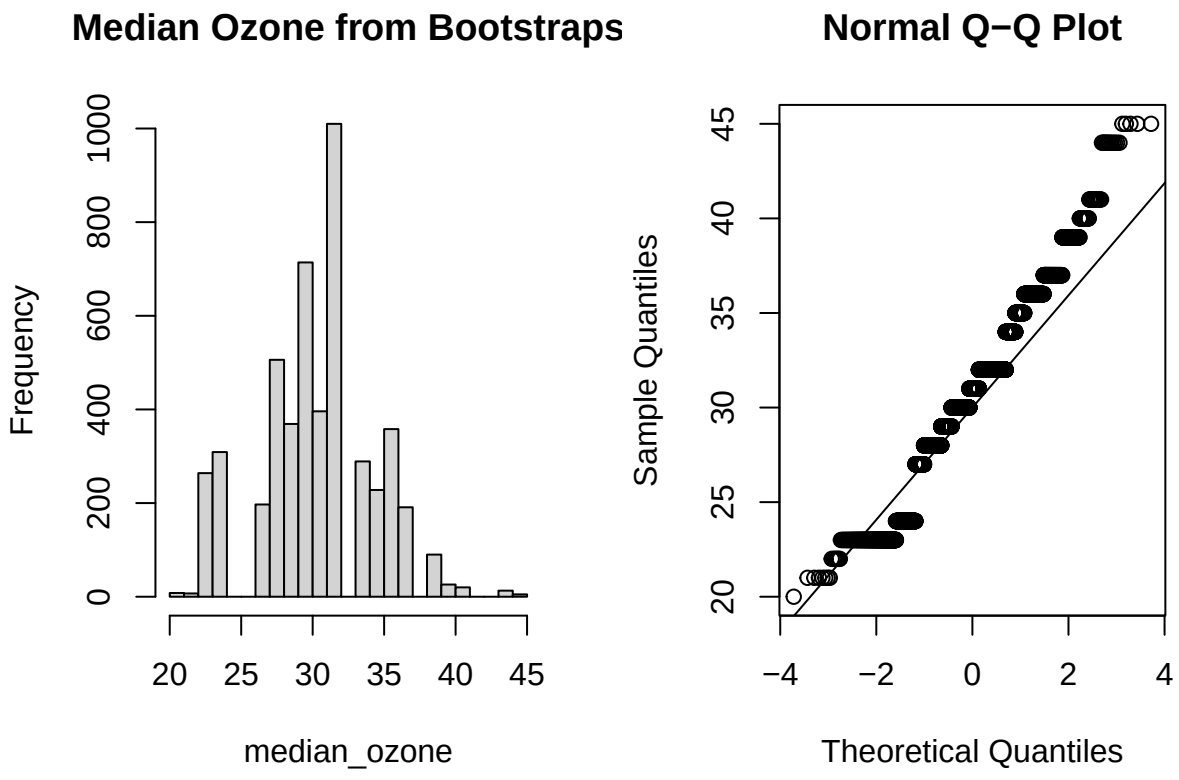



```
base_model
```

```
##
## Call:
## lm(formula = Ozone ~ Solar.R + Wind + Temp - 1, data = complete_data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -40.675 -15.446  -5.526  13.479  88.822
##
## Coefficients:
##         Estimate Std. Error t value Pr(>|t|)
## Solar.R  0.06306    0.02387   2.641  0.00948 **
## Wind    -4.59884    0.48653  -9.452 8.21e-16 ***
## Temp     0.98525    0.08739  11.275  < 2e-16 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 21.84 on 108 degrees of freedom
## Multiple R-squared:  0.8383, Adjusted R-squared:  0.8338
## F-statistic: 186.7 on 3 and 108 DF,  p-value: < 2.2e-16
```

```
# Every linear model was fit without the intercept to improve model fit and
# interpretability. All three parameter estimates follow a very normal shape,
# which is to be expected from the uniform sampling of the original data set. The
```

```
# minimum and maximum for both Solar.R and Temperature being above (or near) 0
# suggests very strongly that they have a positive correlation with Ozone. When
# either variable increases, ozone increases. The opposite is true for Wind, where
# both its extremum are below 0, suggesting wind reduces ozone. All medians were
# very close to their respective means, shown by the symmetry of each distribution.
# Additionally, each mean value is very close to the estimate from the base
# model, suggesting that the significant effects of each variable are usually
# retained after bootstrapping. This is great evidence that the very significant
# effects of wind and temperature on ozone are "true", and the lower-in-magnitude
# but still significant effect of Solar.R on ozone is true.
# It should be noted that some estimates may be altered by collinearity between
# explanatory variables. Though the correlation matrix reveals relatively weak
# relationships between any pair of them, this correlation could change
# dramatically if the "right" rows from the data are selected via bootstrap.
# Additional analysis could consider variance inflation factors to account for
# this possibility.

### Examining median of response variable
par(mfrow=c(1,2))
hist(median_ozone,main="Median Ozone from Bootstraps",breaks=20)
qqnorm(median_ozone)
qqline(median_ozone)
```




```
# The median ozone has heavy tails, as revealed by the QQ-plot. There are also
# some gaps in the histogram, namely around 25, 32.5, and 37.5. This is likely
```

```r
# just a result of the bootstrapping process, and would vary with different runs
# of the analysis.
cbind(Boot=mean(median_ozone),Base=quantile(complete_data$Ozone,0.5))
```

```
##        Boot Base
## 50% 30.7882   31
```

```r
# The average median of the bootstrapped samples and the median of the base
# data are almost identical.

# A good idea for further analysis would be bootstrapping with more advanced
# missing data techniques - i.e. not complete-case analysis. It would be
# interesting to examine if bootstrapping leads to any differences with mean or
# regression imputation, non-response weighting, etc.
rm(list = ls())
```

Lecture 5 Problems

1.

In your real dataset, try to use software package to apply the EM algorithm to the multivariate normal model and report the results (you can have one variable missing or more than one variable missing)

```r
data("airquality")
em_data <- airquality[,1:4] %>% as.matrix()
s <- prelim.norm(em_data)
estimates <- em.norm(s) # em algorithm, multivariate normal
```

```
## Iterations of EM:
## 1...2...3...4...5...6...7...
```

```r
results <- getparam.norm(s,estimates,corr=T) # $sdv

## Mean and Variance Comparison
rbind(EM_Mean=results$mu,
      Base_Mean=colMeans(airquality[,1:4],na.rm=T),
      EM_Var=results$sdv^2,
      Base_Var=apply(airquality[,1:4],2,var,na.rm=T))
```

```
##                Ozone   Solar.R      Wind     Temp
## EM_Mean     41.87100  184.8471  9.957516 77.88235
## Base_Mean   42.12931  185.9315  9.957516 77.88235
## EM_Var    1044.04320 8090.6909 12.330417 89.00577
## Base_Var  1088.20052 8110.5194 12.411539 89.59133
```

```r
# Of course, the mean estimates for Wind and Temperature are the same for both
# the EM algorithm and complete-case analysis since there are no missing values.
# Both the Ozone and Solar.R mean estimates are slightly lower for the EM
# algorithm than for the base method. EM also reduces the variance estimate for
```

```
# Ozone and Solar.R, perhaps in a similar way to mean imputation. We'd expect
# the variances for Wind and Temp to match between methods as well - but base R
# uses the sample formulas for variance and standard deviation, while the norm
# package uses the population formulas

## Correlation Comparisons
results$r
```

```
##            [,1]       [,2]       [,3]       [,4]
## [1,]  1.0000000  0.3242646 -0.5696893  0.6874234
## [2,]  0.3242646  1.0000000 -0.0548922  0.2805526
## [3,] -0.5696893 -0.0548922  1.0000000 -0.4579879
## [4,]  0.6874234  0.2805526 -0.4579879  1.0000000
```

```
cor(airquality[,1:4] %>% na.omit())
```

```
##              Ozone    Solar.R       Wind       Temp
## Ozone    1.0000000  0.3483417 -0.6124966  0.6985414
## Solar.R  0.3483417  1.0000000 -0.1271835  0.2940876
## Wind    -0.6124966 -0.1271835  1.0000000 -0.4971897
## Temp     0.6985414  0.2940876 -0.4971897  1.0000000
```

```
# Every variable relationship is attenuated - brought closer to zero - by the
# EM algorithm. The most notable relationships remain the negative relationship
# between wind and Ozone, and the positive relationship between Temp and Ozone.
# Their is somewhat of a negative correlation between Wind and Temp, though this
# is also attenuated. All other correlation pairs remain relatively small, or
# essentially nothing in the case of Wind and Solar.R

rm(list=ls())
```

Lecture 6 Problems

1.

(Optional) Use the SAS or R code for Example 6.4, can you find the posterior mean, variance, and 95% credible interval for β_0 (the intercept) and also report the mean of the imputed values at the 1st iteration?

```
bayes_missing <- function(n,mu_x,var_x,var_eps,beta0,beta1,alpha0,alpha1,
                          n_chains,iterations,burn,seed=78){
  # Number of values to generate, distributional parameters
  # Betas for generation of response variable
  # Alphas for MAR missingness mechanism
  # Stan parameters - NOTE: large values for chains and iter take a LONG time

  # generate data
  x <- mu_x + sqrt(var_x)*rnorm(n)
  error <- rnorm(n)
  y <- beta0 + beta1*x + error
  miss_indi <- rbinom(n=n, size=1,
```

```
                    prob=exp(alpha0+alpha1*x)/(1+exp(alpha0+alpha1*x)))
y[miss_indi==1] <- NA # replace "missing" values with NA

# data objects for model
n_miss <- sum(is.na(y))
n_obs <- n-n_miss
y_obs <- y[!is.na(y)] # observed y
x_obs <- x[!is.na(y)] # x's with observed y
x_miss <- x[is.na(y)] # x's with missing y

# model code, I tried to make this a parameter but R was being weird
linear_model <- "data{
  int<lower=0> n_obs;
  int<lower=0> n_miss;
  vector[n_obs] y_obs;
  vector[n_obs] x_obs;
  vector[n_miss] x_miss;
}
parameters{
  real beta0;
  real beta1;
  real<lower=0> sigma;
  vector[n_miss] y_miss;
}
model{
  y_obs ~ normal(beta0+beta1*x_obs, sigma); // normal likelihood for y_obs
  y_miss ~ normal(beta0+beta1*x_miss, sigma); // normal likelihood for y_mis
  beta0 ~ normal(0, 1000); // prior for beta0
  beta1 ~ normal(0, 1000);  // prior for beta1
  sigma ~ exponential(0.0008); // prior for sigma
}
"
# generate values for parameters
missing_data_sim <- stan(model_code = linear_model,
                         data=list(n_obs,n_miss,y_obs,x_obs,x_miss),
                         chains=n_chains,iter=iterations,warmup=burn,seed=seed)
generations <- missing_data_sim %>% as.data.frame()
generated_data <- generations[,-c(1,2,3,ncol(generations))] # only y values

# calculate summary stats and save in a list to return
summary_stats <- list(Means=colMeans(generations[,1:2]),
                      Var=apply(generations[,1:2],2,var),
                      Lower_95=apply(generations[,1:2],2,quantile,0.025),
                      Upper_95=apply(generations[,1:2],2,quantile,0.975))
# plot diagnostics, save in a list to return
plots <- list(Beta0_Trace=mcmc_trace(missing_data_sim, pars='beta0'),
              Beta0_Hist=mcmc_hist(missing_data_sim, pars='beta0'),
              Beta1_Trace=mcmc_trace(missing_data_sim, pars='beta1'),
              Beta1_Hist=mcmc_hist(missing_data_sim, pars='beta1'))

# return summary stats, plots, and all generated values (for future use)
return(list(Summary_Stats=summary_stats,
            Plots=plots,
```

```
              Generated_Values=list(All=generations,
                                    Missing_Data=generated_data)))
}

bayes_results1 <- bayes_missing(1000,1,1,1,-2,1, # base data generation
                                -1.4,1, # missingness generation parameters
                                1,10000,burn=2000) # stan parameters
```

```
##
## SAMPLING FOR MODEL 'anon_model' NOW (CHAIN 1).
## Chain 1:
## Chain 1: Gradient evaluation took 0.000241 seconds
## Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 2.41 seconds.
## Chain 1: Adjust your expectations accordingly!
## Chain 1:
## Chain 1:
## Chain 1: Iteration:    1 / 10000 [  0%]  (Warmup)
## Chain 1: Iteration: 1000 / 10000 [ 10%]  (Warmup)
## Chain 1: Iteration: 2000 / 10000 [ 20%]  (Warmup)
## Chain 1: Iteration: 2001 / 10000 [ 20%]  (Sampling)
## Chain 1: Iteration: 3000 / 10000 [ 30%]  (Sampling)
## Chain 1: Iteration: 4000 / 10000 [ 40%]  (Sampling)
## Chain 1: Iteration: 5000 / 10000 [ 50%]  (Sampling)
## Chain 1: Iteration: 6000 / 10000 [ 60%]  (Sampling)
## Chain 1: Iteration: 7000 / 10000 [ 70%]  (Sampling)
## Chain 1: Iteration: 8000 / 10000 [ 80%]  (Sampling)
## Chain 1: Iteration: 9000 / 10000 [ 90%]  (Sampling)
## Chain 1: Iteration: 10000 / 10000 [100%]  (Sampling)
## Chain 1:
## Chain 1:  Elapsed Time: 2.037 seconds (Warm-up)
## Chain 1:                4.776 seconds (Sampling)
## Chain 1:                6.813 seconds (Total)
## Chain 1:
```

```
# posterior mean, variance, 95% credible intervals for beta0, beta1
# the results are pretty accurate considering the true values are -2 and 1,
# low variance is also a good sign of convergence (and confidence in the values)
bayes_results1$Summary_Stats
```

```
## $Means
##     beta0     beta1
## -2.068426  1.090307
##
## $Var
##       beta0       beta1
## 0.002809529 0.002355519
##
## $Lower_95
##      beta0      beta1
## -2.1711015  0.9956477
##
## $Upper_95
```

```
##     beta0     beta1
## -1.965378  1.185841
```

```
# mean of y at first generation (first row)
bayes_results1$Generated_Values$Missing_Data[1,] %>% unlist() %>% mean()
```

```
## [1] -0.5943543
```